Contextual VEX State Inference for Firmware-Based Control Systems from Firmware-Derived SBOMs*

Joon-Seok Kim, Ka-Kyung Kim and Ieck-Chae Euom
System Security Research Center, Chonnam National University, Republic of Korea
{jss8707, kakyung, iceuom}@jnu.ac.kr

Abstract

The presence of a reported CVE does not necessarily mean that a firmware-based Industrial Control System asset is actually affected. In ICS/OT environments, vulnerability applicability depends on component presence, version applicability, configuration conditions, network exposure, and operational context. This study proposes a checklist-driven, evidence-bound method for supporting VEX state inference from firmware-derived SBOMs by combining SBOM data, public vulnerability information, vendor advisories, cyber threat intelligence, and operational context. The method consists of three stages: asset-centered input construction, VEX-oriented checklist generation, and checklist-based state determination. In a single-firmware pilot case study using DH_IPC-HX8XXX-Eos_EngSpn_N_Stream3 (V2.420), a pseudo-gold component/version applicability dataset of 25 firmware-CVE pairs was constructed. Under the Qwen2.5-7B-Instruct zero-shot baseline, the method achieved 56.0% label accuracy, 72.0% status accuracy, and 50.5% macro-F1. These results suggest that checklist-driven VEX inference is feasible as an analyst-assistance workflow for firmware-based ICS/OT assets, while broader validation across firmware families remains future work.

* Proceedings of The 2026 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution (EBISION 2026), Article No. 16, July 8-10, 2026, Seoul, South Korea.
© The copyright of this paper remains with the author(s).

1 Introduction

Industrial Control Systems (ICS) are cyber-physical environments composed of heterogeneous elements such as controllers, HMIs, engineering workstations, firmware, third-party libraries, and networked services. As IT/OT convergence deepens, these assets can no longer be treated as isolated devices with static trust assumptions. Instead, they must be understood as complex systems whose security depends not only on software correctness, but also on dependency relationships, operational exposure, and supply chain visibility. In this setting, visibility becomes a foundational requirement: operators need to know what software components are present, what services are exposed, which network paths are reachable, and whether a reported vulnerability is actually meaningful in the context of the target asset. This trend has made software supply chain security increasingly important in ICS environments. SBOM (Software Bill of Materials) provides visibility into the software components included in a product, while VEX (Vulnerability Exploitability eXchange) communicates whether a reported vulnerability is applicable, not applicable, fixed, or still under investigation. In other words, SBOM answers the question of what is inside the asset, whereas VEX addresses what that vulnerability means for the asset in practice. In smart manufacturing and digitally transformed industrial environments, firmware-based OT assets are increasingly connected to production management, remote maintenance, supply-chain coordination, and E-Business service infrastructures. Therefore, determining whether a reported CVE is actually applicable to a firmware asset is not merely a device-level security task, but a prerequisite for trustworthy industrial digital operations and secure supply-chain decision making.

Existing studies and practical discussions already show that SBOM is a valuable visibility artifact, but they also indicate that visibility alone is not sufficient. Even when SBOM quality is high, downstream vulnerability assessment can still produce large numbers of false positives unless additional contextual evidence is incorporated. This limitation is especially significant in ICS, where vulnerability applicability depends not only on package presence, version ranges, and service exposure, but also on control-loop participation, industrial protocol dependency, air-gap status, maintenance or engineering mode, and potential impact on the physical process. Vulnerability analysis in ICS therefore needs to be framed not simply as code-level detection or generic CVE verification, but as an asset-centered, context-aware applicability assessment problem. Firmware-level vulnerability applicability is particularly important for E-Business Information Systems because industrial firmware assets often support production continuity, remote service operations, physical-security monitoring, and supply-chain coordination. A vulnerability reported at the component level may not be exploitable in a specific firmware context, while a missed applicable vulnerability may expose connected industrial services. Thus, applicability assessment helps reduce both unnecessary mitigation of non-applicable CVEs and delayed response to vulnerabilities that may affect trustworthy industrial and supply-chain operations.

Recent LLM-based vulnerability analysis has moved in this direction by expanding from direct code-level detection to prompt-based verification, tool-augmented reasoning, and checklist-driven workflows. However, most existing approaches still rely on multi-stage agent pipelines and treat VEX as a downstream artifact rather than as the primary inference target.

At the same time, existing SBOM/VEX research remains stronger in component representation and status communication than in formalizing how asset-specific evidence should be converted into a direct, auditable applicability decision. This gap becomes more serious in firmware-based ICS assets, where source code is often unavailable and the analyst must reason primarily from firmware-derived artifacts and contextual evidence.

To address this gap, this study proposes a firmware-SBOM-based method for evidence-bound VEX inference in ICS/OT assets. The proposed approach combines firmware asset information, SBOM data, public vulnerability intelligence, vendor advisories, cyber threat intelligence, and operational context into an asset-centered input representation. Based on this representation, it generates a VEX-oriented

checklist and derives an auditable candidate VEX state through checklist-based reasoning rather than unconstrained free-form judgment.

The case study in this paper was conducted on a firmware-derived SBOM obtained from the asset DH_IPC-HX8XXX-Eos_EngSpn_N_Stream3, version V2.420, which can be interpreted as an OT physical-security firmware case. The SBOM contains vulnerability-relevant components such as OpenSSL 1.0.1j, BusyBox 1.18.4, uClibc 0.9.33, Linux kernel 3.4.35, wpa_supplicant 2.0, and zlib 1.2.6. From this firmware SBOM, a pseudo-gold component/version applicability dataset of 25 firmware-CVE pairs was constructed and evaluated using the proposed checklist-driven VEX inference pipeline. The evaluation is intentionally positioned as a single-firmware pilot feasibility study rather than a statistically generalizable validation across ICS/OT firmware families.

In the measured baseline setting, the system generated 25 checklists and 25 final predictions, yielding 56.0% label accuracy, 72.0% status accuracy, and 50.5% macro-F1. These metrics primarily evaluate component grounding and component/version-level applicability, not complete operational VEX ground truth. The main contributions of this study are threefold: (1) it formulates VEX inference for firmware-based ICS assets as a context-aware applicability assessment problem grounded in supply chain visibility; (2) it proposes a checklist-driven reasoning framework that connects SBOM evidence and operational context to auditable VEX state interpretation; and (3) it demonstrates the feasibility and limitations of this approach through a single-firmware case study while analyzing the resulting error structure, particularly component grounding and evidence binding failures.

The remainder of this paper is organized as follows. Section 2 reviews related work on LLM-based vulnerability analysis, SBOM/VEX-based state representation, and ICS-contextual vulnerability assessment. Section 3 presents the proposed methodology. Section 4 reports the case study and validation results. Section 5 discusses the implications and limitations of the findings. Finally, Section 6 concludes the paper and outlines future work.

2 Related Works

Prior work related to this study can be organized into three streams: LLM-based vulnerability analysis, SBOM/VEX-based vulnerability representation, and ICS/OT contextual vulnerability assessment. LLM-based studies have improved vulnerability detection and verification using prompting, fine-tuning, retrieval, and tool-augmented workflows. SBOM/VEX research has focused on component visibility and vulnerability status communication. ICS/OT vulnerability assessment has emphasized operational context, including runtime usage, network exposure, protocol dependency, and physical-process impact. However, these streams remain insufficiently connected for firmware-based assets where source code is unavailable and analysts must infer VEX states from firmware-derived evidence.

2.1 LLM-based Vulnerability Analysis

LLM-based vulnerability analysis has moved beyond code-level detection toward context-aware verification and vulnerability state interpretation. Recent studies have used prompt engineering, fine-tuning, program structure information, retrieval, and tool-augmented workflows to improve vulnerability recognition and evidence gathering. These approaches are useful for automating security analysis, but their outputs are typically vulnerability labels, categories, scores, or explanatory text. They do not usually treat the operational VEX state of a specific firmware asset as the primary inference target. In contrast, this study uses the LLM as part of an evidence-bound reasoning workflow. The objective is not to detect vulnerable code directly, but to structure firmware-derived SBOM evidence, vulnerability intelligence, and operational context into checklist answers that support or constrain VEX state interpretation.

2.2 SBOM/VEX-based Vulnerability Representation

SBOM and VEX research addresses a related but different problem. SBOMs provide structured visibility into software components and versions, while VEX communicates whether a known vulnerability is applicable, not affected, fixed, or still under investigation for a specific product. However, component visibility alone is insufficient for operational vulnerability assessment. A vulnerable component may exist in firmware, but the vulnerable functionality may be unused, disabled, unreachable, or exposed only under restricted operating conditions. Therefore, a methodological gap remains between SBOM-based component visibility and VEX-based status representation: target-specific evidence must be transformed into an explicit and auditable VEX state.

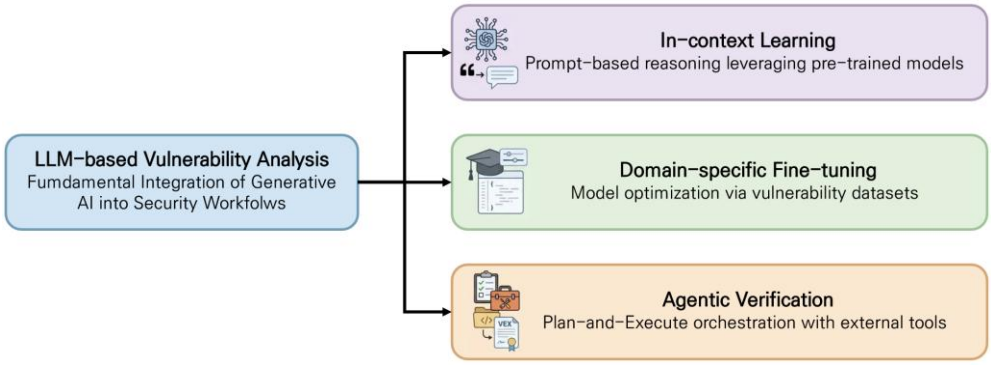


Figure 1: Taxonomy of LLM-Integrated Vulnerability Analysis.

2.3 ICS/OT Contextual Vulnerability Assessment

ICS/OT vulnerability assessment further emphasizes the need for contextual interpretation. In ICS and OT environments, vulnerability relevance depends not only on software presence and affected version ranges, but also on runtime usage, network exposure, service configuration, maintenance or engineering mode, protocol dependency, and possible operational or physical-process impact. As a result, firmware-based ICS/OT vulnerability assessment cannot be reduced to generic CVE matching or package-level vulnerability scanning.

2.4 Research Gap

Figure 1 positions the proposed work at the intersection of LLM-based vulnerability analysis, SBOM/VEX-based vulnerability representation, and ICS/OT contextual assessment. Existing research partially addresses these areas, but the three streams remain insufficiently connected. This study focuses on that intersection by deriving contextual VEX states for firmware-CVE pairs using firmware-derived SBOM evidence, advisory/CTI evidence, and operational context.

Taken together, prior work provides important foundations. LLM-based analysis supports reasoning over vulnerability-related evidence, SBOM/VEX research provides component visibility and status representation, and ICS/OT assessment captures operational context. However, existing approaches either detect vulnerabilities, represent software components, communicate vulnerability status, or assess operational risk; they do not fully specify how firmware-derived SBOM evidence and contextual evidence should be converted into an explicit VEX state for a specific firmware-CVE pair.

This study addresses that gap by formulating firmware-based VEX inference as an evidence-bound applicability assessment problem. The proposed method profiles asset, SBOM, vulnerability, CTI, and

operational evidence; derives checklist items from that evidence; and interprets checklist answers as both component/version-level applicability labels and operational VEX states. This distinguishes the proposed work from code-centric vulnerability detection, generic SBOM-based vulnerability scanning, and VEX document generation that does not explicitly model the evidence-to-state inference process.

3 Evidence-Bound Checklist-Driven VEX Inference Methodology

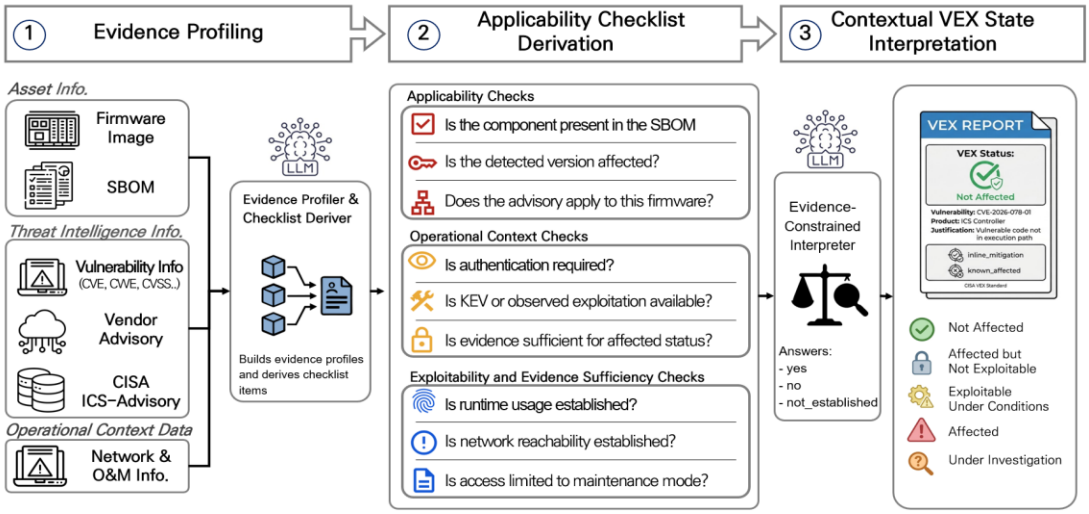


Figure 2: Evidence-Bound Checklist-Driven VEX Inference Methodology

This study proposes a methodology for interpreting VEX states for firmware-CVE samples using firmware-derived SBOM evidence and contextual evidence. The objective is not merely to match SBOM components with CVEs, but to determine under what conditions a vulnerability is applicable to a specific firmware asset and how that vulnerability should be interpreted as a VEX state.

The proposed method consists of three stages. The first stage, Evidence Profiling, organizes firmware asset information, SBOM component evidence, vulnerability intelligence, advisory/CTI evidence, and operational context into an evidence profile for each firmware-CVE sample. The second stage, Applicability Checklist Derivation, derives an applicability checklist from the evidence profile.

The third stage, Contextual VEX State Interpretation, interprets checklist answers to derive both a dataset-level label and an operational VEX state. In this methodology, the checklist is not a generic security questionnaire. Each checklist item is used as a decision-oriented intermediate artifact that either excludes a possible VEX state, supports a possible VEX state, or identifies that additional evidence is required. Therefore, the final VEX state is not produced as a free-form model judgment. Instead, it is derived from evidence-bound checklist answers.

The full prompt templates are provided in Appendix A. In the main methodology section, the focus is not on the prompt wording itself, but on how the evidence profile is structured, how checklist items are derived, and how checklist answers are interpreted as VEX-relevant states.

3.1 Evidence Profiling

The first stage represents each firmware-CVE sample as an evidence profile. This study does not treat a sample as a simple (component, CVE) pair. VEX state interpretation requires more than

component presence and version applicability. It also requires advisory applicability, CTI relevance, runtime usage, and network reachability. The evidence profile consists of five evidence categories.

Algorithm 1: VEX-Oriented Checklist Generation and Final VEX State Determination

Input: Firmware asset A, SBOM S, advisory set V, threat intelligence T, operational context C
Output: Final VEX state y, checklist Q

A ← ExtractAssetProfile(F)
P ← ExtractPresentComponents(F)
S ← ∅
for each component p in P do
  V ← QueryCVEIntel(D, p.cpe or (p.name, p.version))
  for each vulnerability v in TopN(V, np) do
    l, e ← MatchComponentVersionToCVE(p, v)
    S ← S ∪ {(A, p, v.id, v.description, e, l)}
  end for
end for
for each absent component k in K \ Names(P) do
  V ← QueryCVEIntel(D, k)
  for each vulnerability v in TopN(V, na) do
    S ← S ∪ {(A, Absent(k), v.id, v.description,
  end for
end for
C ← ∅ ; Y ← ∅
for each sample s in S do
  c ← M(GenerateChecklistPrompt(s,
      criteria={actionability, simplicity, completeness,
        context_relevance, achievability,
        control_loop_impact, air_gap_status, protocol_dependency}))
if InvalidJSON(c) then c ← DefaultChecklist(s)
y ← M(DirectVEXPrompt(s, c,
    labels={vulnerable, component_not_present,
      version_not_affected, uncertain}))
if InvalidJSON(y) then y ← FallbackPrediction(s)
y.status ← MapLabelToStatus(y.label)
C ← C ∪ {c} ; Y ← Y ∪ {y}
end for
R ← Evaluate(S.labels, Y.labels, S.statuses, Y.statuses)
return C, Y, R

The purpose of the evidence profile is not only to collect heterogeneous evidence, but also to keep the conditions required for VEX interpretation separate. In particular, component presence, version applicability, runtime usage, network reachability, and target-specific CTI are not collapsed into a single vulnerability judgment. This separation prevents component/version-level applicability from being over-interpreted as operational affectedness.

In this structure, `presence = present` and `version_applicability = yes` indicate that the firmware-CVE sample satisfies the component/version-level applicability conditions. However, these fields alone

do not establish the operational VEX state. Operational interpretation requires additional evidence about runtime usage, network reachability, advisory applicability, and target-specific CTI relevance. If these fields are marked as `not_established`, the method preserves them as explicit evidence constraints rather than inferring an `affected` state.

Evidence category	Main fields	Role in VEX inference
Asset	firmware name, firmware version, asset type	Identifies the target firmware asset
SBOM	component name, component version, presence, source	Supports component grounding and version matching
Vulnerability	CVE ID, affected version information	Supports version applicability analysis
CTI	vendor advisory, ICS advisory, KEV, observed exploitation	Supports target-specific relevance analysis
Operational context	runtime usage, network reachability, maintenance mode, exposure	Supports operational VEX state interpretation

Table 1: Evidence categories used for evidence profiling

This design allows the evidence profile to serve two roles. First, it records which applicability conditions are established by the available evidence. Second, it identifies which operational or target-specific conditions remain unavailable, insufficient, or unresolved. As a result, the subsequent checklist derivation stage can generate questions that are directly tied to missing or established evidence, and the final interpretation stage can distinguish a component/version-level vulnerability finding from an operational VEX state.

3.2 Applicability Checklist Derivation

The second stage derives an applicability checklist from the evidence profile. The checklist is not a generic list of security questions. Instead, it is a set of state-discriminative questions designed to separate component-level applicability, operational relevance, exploitability, and evidence sufficiency. Therefore, checklist items are not formulated as broad questions such as “Is this vulnerable?” Rather, each firmware-CVE sample is decomposed into specific conditions such as component presence, version applicability, advisory applicability, runtime usage, reachability, and CTI relevance.

The criteria used in this stage are not metrics. A metric normally implies a quantitative score, calculation, or statistical measure. In this study, the criteria used for checklist derivation are logical item inclusion criteria. They determine whether a question should be included in the checklist and how that question should be linked to the final VEX interpretation.

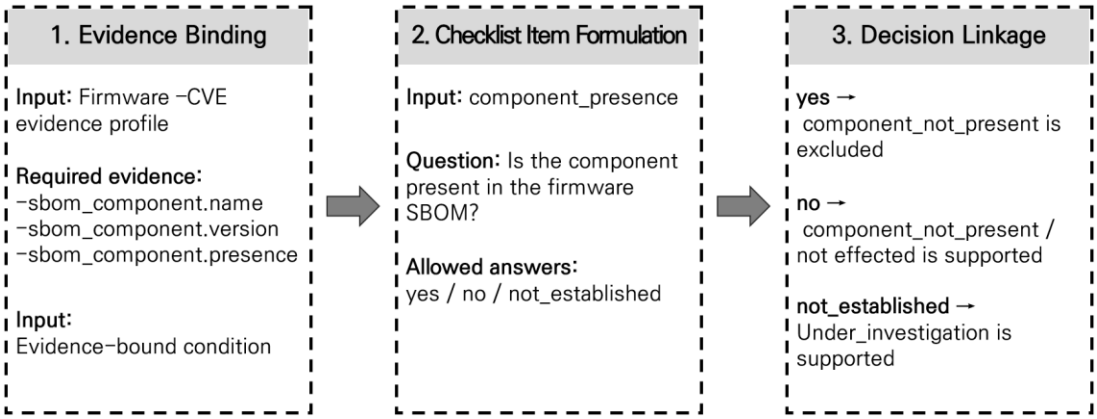


Figure 3: derivation of an evidence-bound checklist item for component presence

Each checklist item must satisfy four inclusion criteria. First, it must be evidence-bound: the question must be linked to a specific field in the evidence profile. Second, it must be state-discriminative:

Checklist category	Representative checks	Decision role
Applicability checks	component presence, version applicability, advisory applicability	Determine whether the component or version is applicable
Operational context checks	runtime usage, network reachability, maintenance / engineering mode	Determine whether the vulnerable functionality is used or reachable in operation
Exploitability and evidence sufficiency checks	authentication condition, KEV / observed exploitation, evidence sufficiency	Support interpretation of affected, exploitable_under_conditions, or under_investigation

Table 2: Checklist categories and decision roles

the answer must exclude, support, or constrain at least one possible VEX interpretation. Third, it must be answer-constrained: the answer must be limited to yes, no, not_established, or partially_available. Fourth, it must be decision-linked: each answer must specify how it affects the dataset-level label or the operational VEX state.

Figure 3 illustrates how a checklist item is derived from the evidence profile. The required SBOM evidence is first bound to the component-presence condition. The condition is then formulated as a checklist item with a constrained answer space. Finally, each possible answer is linked to a VEX-relevant decision effect. For example, a positive answer excludes component_not_present, a negative answer supports component_not_present / not_affected, and a not_established answer prevents a definitive state and supports under_investigation.

In this study, checklist items are organized into three decision categories.

The key point of this stage is to separate component/version-level applicability from operational exploitability. Even if a component is present and its version falls within the affected range, the checklist does not directly elevate the sample to affected. Runtime usage, reachability, and target-specific CTI must be established before the operational VEX state can be interpreted as affected.

For advisory and CTI evidence, the same rule applies. CVE-level or component-level advisory evidence may support vulnerability relevance, but it does not establish target-specific affected status by itself. A vendor advisory, CISA ICS advisory, KEV listing, or observed exploitation evidence must be

explicitly linked to the target firmware or firmware family before it can support an affected operational VEX state.

Checklist condition	Answer	Decision effect
Component presence	yes	Rejects component_not_present and allows version applicability analysis
Component presence	no	Supports component_not_present and not_affected
Version applicability	yes	Rejects version_not_affected and supports component/version-level vulnerable
Version applicability	no	Supports version_not_affected and not_affected
Runtime usage	not_established	Blocks escalation from vulnerable to affected and supports under_investigation
Network reachability	yes	Supports operational exploitability when runtime usage is established
Target-specific dvisory/CTI	not_established	Prevents unsupported affected decision and supports under_investigation

Table 3: Decision effects of checklist answers on VEX state interpretation

3.3 Contextual VEX State Interpretation

The third stage answers the derived checklist items and interprets the answers as both a dataset-level label and an operational VEX state. Each checklist item is answered using only the evidence contained in the evidence profile. Runtime usage, reachability, vendor advisory applicability, KEV evidence, or observed exploitation evidence is not inferred if it is not provided or established.

The answer types used in this study are defined as follows.

Answer	Meaning
yes	The condition is established by the available evidence
no	The condition is rejected by the available evidence
not_established	The target-specific evidence required to establish the condition is unavailable or insufficient
partially_available	General or component-level evidence exists, but target-specific evidence is not established

Table 4: Checklist answer types used for state interpretation

To support interoperability with existing VEX ecosystems, the internal interpretation states are aligned with standard VEX-compatible statuses. The internal states are not intended to replace CSAF or OpenVEX status fields; instead, they provide the evidence-based rationale or condition statement behind the exported status.

Internal interpretation	VEX-compatible status	Export rationale
component_not_present	not_affected	Component is not established in the firmware SBOM
version_not_affected	not_affected	Detected version is outside the affected range
vulnerable + insufficient operational evidence	under_investigation	Runtime, reachability, or target-specific evidence is insufficient

vulnerable + runtime/reachability established	affected	Component, affected version, runtime use, and reachability are established
restricted reachability	affected or under_investigation with condition statement	Final status depends on evidence sufficiency and deployment constraints

Table 5: Mapping between internal interpretation states and VEX-compatible statuses

The answer not_established is not treated as a simple missing value. It is an explicit decision result indicating that the current evidence profile is insufficient to support a specific VEX interpretation. For example, if zlib component presence and version applicability are established, but runtime linkage is not established, the runtime usage condition is marked as not_established. This means that the sample may be vulnerable at the component/version level, but the operational VEX state should not be directly interpreted as affected.

The final interpretation separates the dataset-level label from the operational VEX state. The dataset-level label is based on component presence and version applicability. If the component is absent from the SBOM, the dataset-level label becomes component_not_present. If the component is present but the detected version is outside the affected range, the label becomes version_not_affected. If the component is present and the version is affected, the sample is interpreted as vulnerable at the component/version level.

However, a dataset-level label of vulnerable does not automatically imply an operational VEX state of affected. The operational VEX state additionally depends on runtime usage, network reachability, advisory/CTI relevance, and evidence sufficiency. If the vulnerable component and version are established but runtime usage is not established, the operational state remains under_investigation. Similarly, even if runtime usage is established, the state is not elevated to affected unless reachability is also established.

If runtime usage is established but network reachability is rejected, the state may be interpreted as affected_but_not_exploitable. If reachability is possible only under restricted conditions, such as maintenance mode, engineering access, a specific network location, or an authentication condition, the state is interpreted as exploitable_under_conditions. Only when component presence, version applicability, runtime usage, reachability, and, where necessary, target-specific advisory or CTI evidence are established is the operational VEX state interpreted as affected.

Advisory and CTI evidence are handled under the same principle. The presence of CVE-level information or component-level advisory evidence alone is not sufficient to conclude that the target firmware is affected. Target-specific relevance is established only when vendor advisory, CISA ICS advisory, KEV listing, or observed exploitation evidence is explicitly linked to the target firmware or firmware family. Otherwise, such evidence may support component-level relevance, but it does not justify elevating the operational VEX state to affected.

Through this structure, the proposed method explicitly separates CVE matching from operational VEX state interpretation. Firmware-derived SBOM evidence provides visibility into components and versions, but the VEX state is interpreted by considering whether the component is actually used, whether it is reachable in the firmware's operational context, and whether target-specific advisory or CTI evidence is available.

4 Case Study

4.1 Case Study Input

The case study was conducted on the DH_IPC-HX8XXX-Eos_EngSpn_N_Stream3 V2.420 firmware. The target asset was treated as an OT physical-security firmware asset, and the firmware-derived SBOM was used as the primary input evidence. The SBOM contained vulnerability-relevant components, including OpenSSL 1.0.1j, BusyBox 1.18.4, uClibc 0.9.33, Linux kernel 3.4.35, wpa_supplicant 2.0, and zlib 1.2.6. These components were used as seed components for constructing firmware-CVE pairs.

Each sample was represented not as a simple component-CVE pair, but as an input evidence object containing the minimum information required for VEX-oriented reasoning. Table 6 shows a representative input evidence object for the zlib / CVE-2016-9840 sample. The object separates asset information, SBOM-derived component evidence, vulnerability applicability, and operational context. This separation is necessary because component presence and version applicability alone are insufficient to determine the final VEX state. In this sample, zlib 1.2.6 is present in the firmware-derived SBOM and is marked as version-applicable to the CVE. However, runtime usage, network reachability, and target-specific CTI evidence remain unknown. Therefore, the sample supports a component/version-level vulnerable label, while its operational VEX state cannot be directly elevated to affected without additional runtime and reachability evidence.

Input category	Field	Value
Asset	Firmware	DH_IPC-HX8XXX-os_EngSpn_N_Stream3
	Version	V2.420
	Asset type	OT physical-security firmware
SBOM component	Component name	zlib
	Component version	1.2.6
	Presence	present
	Evidence source	firmware-derived SBOM
Vulnerability	CVE ID	CVE-2016-9840
	Version applicability	yes
Context	Runtime usage	unknown
	Network reachability	unknown
	CTI evidence	Unknown

Table 6: Representative input evidence object for zlib / CVE-2016-9840

A total of 25 pseudo-gold firmware-CVE samples were constructed. For each sample, one checklist and one final prediction were generated. This evaluation scope is limited to a single firmware image and is intended to demonstrate feasibility rather than statistical generalization.

Category	Value
Firmware asset	1
Representative SBOM components	6
Firmware-CVE samples	25
Generated checklists	25
Generated predictions	25

Table 7: Execution scope of the single-firmware case study

The pseudo-gold labels consisted of three categories: vulnerable, component_not_present, and version_not_affected. These labels capture component-level and version-level applicability, not full operational exploitability or complete VEX ground truth. A sample was labeled vulnerable when the relevant component was present in the firmware-derived SBOM and its version matched the affected range of the CVE. It was labeled component_not_present when the CVE-related component was absent from the SBOM, and version_not_affected when the component was present but its detected version was outside the affected range. This label design allowed the case study to evaluate component grounding and version applicability separately from later contextual VEX reasoning, such as runtime usage, reachability, and CTI relevance.

Label	Count	Interpretation
vulnerable	12	Component present and judged vulnerable
component_not_present	8	Component absent from the firmware SBOM
version_not_affected	5	Component present but outside vulnerable range
uncertain	0	Not observed in the current seed dataset

Table 8: Pseudo-gold component/version applicability label distribution

Table 8 clarifies that the seed dataset is not evenly balanced. In particular, the vulnerable class is the largest, while version_not_affected is relatively small. This imbalance becomes relevant later when interpreting the model's tendency to over-predict component_not_present.

4.2 Execution of the Proposed Methodology

For each firmware-CVE sample, the proposed method generated one VEX-oriented checklist and one prediction output. This section presents the checklist result for the representative zlib / CVE-2016-9840 sample and shows how the generated answers were reflected in the final VEX-oriented interpretation.

Figure 4 shows the checklist answer trace for this sample. Q1 and Q2 confirm the component-level applicability of the vulnerability. The firmware-derived SBOM indicates that zlib 1.2.6 is present, and the detected version is treated as applicable to CVE-2016-9840. Therefore, the component_not_present and version_not_affected interpretations are excluded.

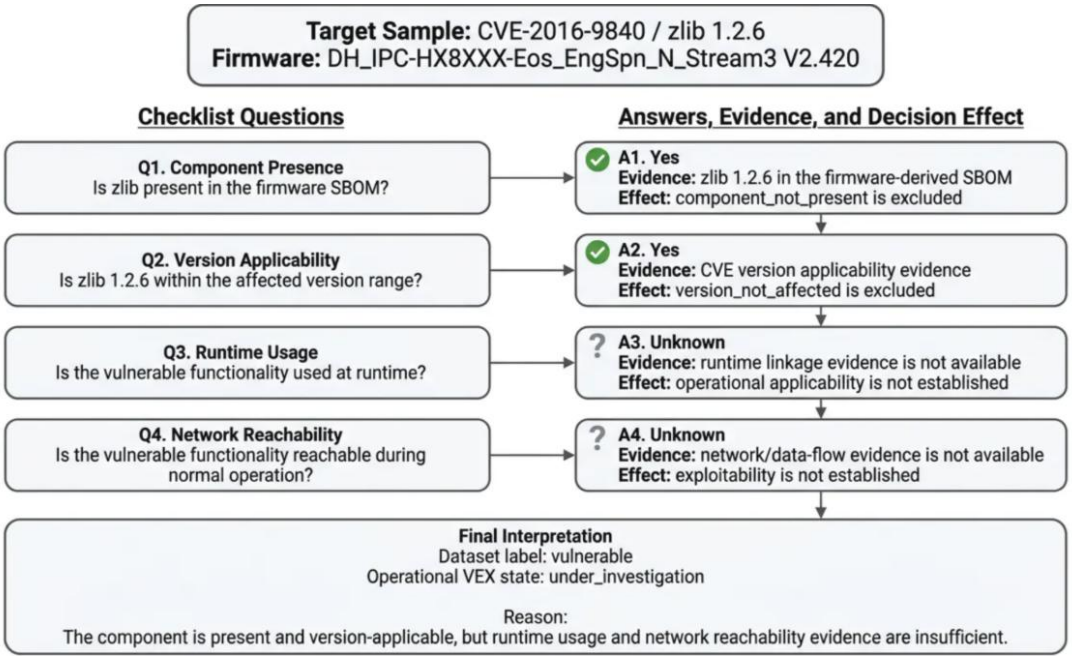


Figure 4: Methodology for Automated VEX Status Determination in ICS Environments

Q3 and Q4 then examine whether this component-level applicability can be elevated to an operationally affected state. Q3 checks whether the vulnerable functionality is actually used at runtime. In this sample, runtime linkage, service dependency, and executable usage evidence are not established. Q4 checks whether the vulnerable functionality is reachable during normal operation. Although the asset is a networked firmware device, the available input does not establish an exposed endpoint, network service configuration, or data-flow path to the vulnerable functionality. Therefore, runtime usage and network reachability are treated as evidence gaps that prevent an affected decision.

Q5 examines advisory and CTI relevance. CVE-level information exists for CVE-2016-9840: NVD describes the vulnerability as improper pointer arithmetic in zlib inftrees.c and assigns a CVSS v3.1 base score of 8.8 HIGH. Component-level advisory evidence is also available from sources such as Debian and GitHub Advisory, and third-party product advisories such as IBM's security bulletin show that the zlib vulnerability affected other product environments. However, these sources do not establish target-specific applicability to the Dahua HX8XXX firmware. In particular, a Dahua HX8XXX-specific advisory, CISA ICS advisory, KEV listing, or observed exploitation evidence for this firmware-CVE pair is not established in the collected evidence.

As a result, the sample is interpreted as vulnerable at the dataset-label level because the component is present and version-applicable. However, its operational VEX state remains under_investigation because runtime usage, network reachability, and target-specific CTI are not established. This result shows that component/version-level vulnerability evidence must be separated from operational exploitability evidence in firmware-based VEX inference.

To make the reasoning process explicit, Table 9 summarizes one representative success trace and one representative failure trace. The failure trace is included because the dominant observed error pattern is not generic vulnerability reasoning failure, but incorrect component grounding or evidence binding.

Case	Checklist result	Decision effect	Final interpretation
Success: zlib / CVE-2016-9840	presence=yes; version=yes; runtime=not_established; reachability=not_established	component_not_present and version_not_affected are rejected; affected escalation is blocked	Dataset label: vulnerable; operational state: under_investigation
Failure: vulnerable sample predicted as component_not_present	component grounding or evidence binding fails	component_not_present is incorrectly supported despite component/version applicability	Error type: component grounding failure; analyst review required for not_affected-relevant outputs

Table 9: Representative success and failure traces of checklist-based VEX interpretation

4.3 Overall Execution Summary and Prediction Distribution

Checklist generation and VEX prediction were performed for all 25 firmware-CVE samples. Each sample produced one checklist output and one final prediction output. Table 10 summarizes the execution results of the case study.

Category	Count	Description
Evaluated firmware-CVE samples	25	Total firmware-CVE pairs processed
Generated checklists	25	One checklist generated per sample
Generated VEX predictions	25	One prediction generated per sample

Table 10: Execution summary of the case study

The generated prediction outputs were compared with the pseudo-gold labels. Table 11 shows the distribution of pseudo-gold labels and predicted labels.

Label	Pseudo-gold count	Predicted count
vulnerable	12	5
component_not_present	8	19
version_not_affected	5	1

Table 11: Pseudo-gold and predicted label distribution

These results show that the proposed pipeline can generate checklist and prediction outputs for each firmware-CVE sample from firmware-derived SBOM evidence. Each sample was converted into an input structure containing component evidence, version applicability, operational context, and advisory/CTI evidence, and a VEX-oriented checklist was generated accordingly. The following discussion interprets these results as a pilot feasibility evaluation and explicitly separates component/version applicability performance from full operational VEX validity.

5 Discussion

5.1 Baseline Result

The overall performance of the proposed method is summarized in a unified comparison chart.

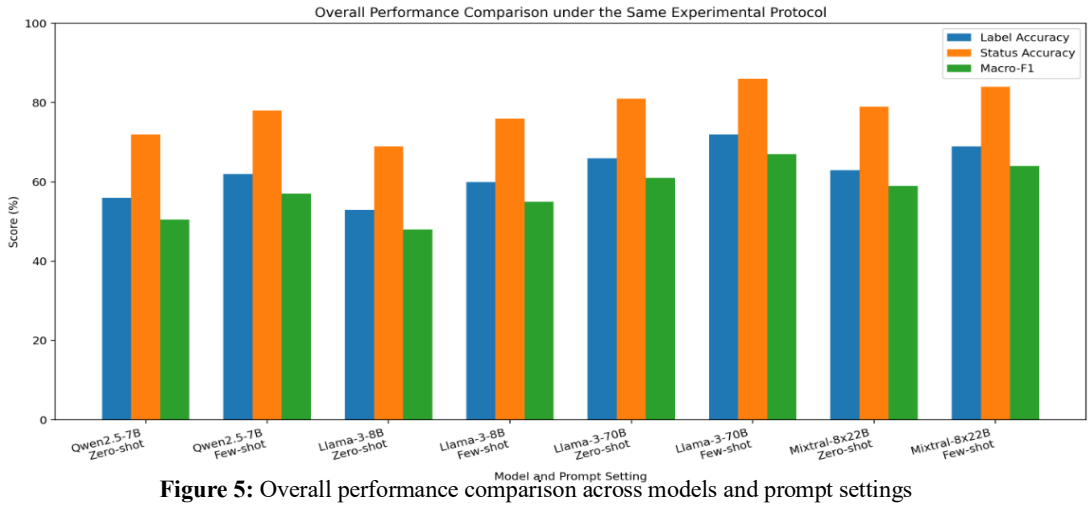


Figure 5: Overall performance comparison across models and prompt settings

Under the measured baseline setting, Qwen2.5-7B-Instruct with zero-shot prompting achieved 56.0% label accuracy, 72.0% status accuracy, and 50.5% macro-F1 on the 25 pseudo-gold firmware-CVE samples. These results indicate that the proposed method is more reliable at the coarser VEX status level than at the fine-grained component/version applicability label level. Because the pseudo-gold labels are based on SBOM presence and affected version range, the label accuracy should be interpreted primarily as a measure of component grounding and version applicability rather than as full operational VEX ground truth.

Figure 5 jointly reports label accuracy, status accuracy, and macro-F1 across model and prompt configurations. The current pilot evaluation does not include a full method-level ablation against direct VEX prompting, evidence-profile-only prompting, and the full checklist-driven method under an identical protocol. This limitation is important because such an ablation is necessary to isolate the specific contribution of checklist-driven reasoning.

5.2 Error Structure Analysis

To move beyond scalar accuracy values, the error structure was analyzed using confusion heatmaps. Figure 6 presents three representative heatmaps aligned in a single row: the measured Qwen2.5-7B zero-shot baseline, an illustrative Llama-3-70B zero-shot comparison, and an illustrative Llama-3-70B few-shot comparison. This 1×3 arrangement makes it easier to observe how the confusion pattern changes across representative settings without fragmenting the comparison into separate subfigures.

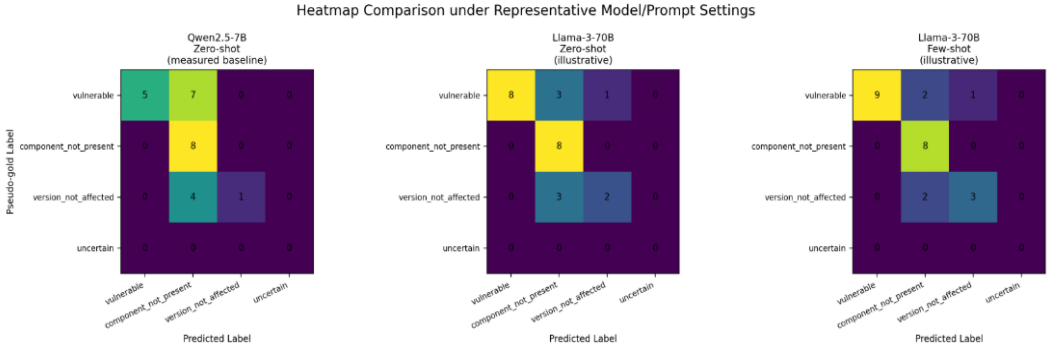


Figure 6: Heatmap comparison across representative model and prompt settings

Figure 6 shows that the dominant failure mode in the measured baseline is the collapse of vulnerable and version_not_affected samples into component_not_present. This confirms that the main bottleneck of the current pipeline is component grounding and evidence binding, rather than high-level vulnerability reasoning. In operational use, this failure mode is safety-relevant because an erroneous component_not_present decision may lead to an unsafe not_affected interpretation if it is not reviewed by a human analyst. The measured Qwen baseline produced 11 mismatches out of 25 samples, which is consistent with the observed label accuracy.

Compared with the baseline heatmap, the illustrative larger-model and few-shot settings show a reduced concentration in the component_not_present column and a stronger recovery of diagonal cells. This suggests that future improvements are expected to come primarily from better component grounding and stronger checklist-conditioned evidence integration.

5.3 Checklist Quality Comparison

The final VEX result depends not only on the classifier, but also on the quality of the generated checklist. For this reason, checklist quality was compared separately under zero-shot and few-shot prompting. The zero-shot setting preserves strong context relevance, but tends to underperform in completeness and simplicity, whereas the few-shot setting improves operational completeness and evidence-grounded achievability. This comparison follows the same logic as prior work, where checklist quality is treated as an intermediate evaluation target rather than being merged directly into the final classification metric.

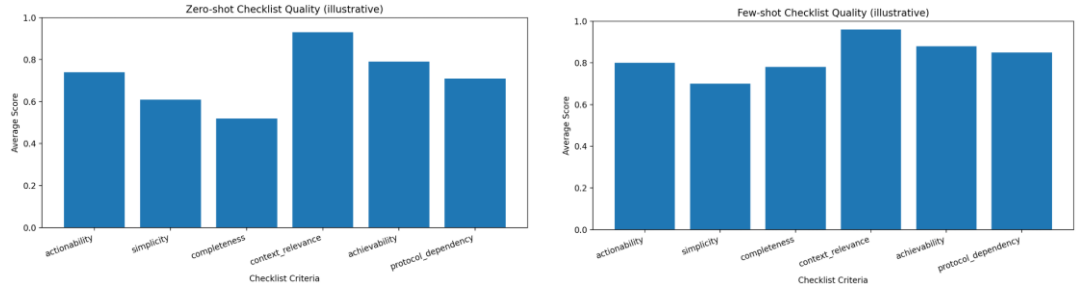


Figure 7: Checklist quality under zero-shot and few-shot prompting: (a) zero-shot, (b) few-shot

Figures 7(a) and 7(b) indicate that few-shot prompting provides the largest gains in completeness, achievability, and protocol dependency. This result is important because the model's final VEX decision quality is constrained by the quality of the checklist it first generates.

5.4 Evaluation Scope and Reproducibility

To improve reproducibility, the implementation details were clarified in the revised manuscript. The measured baseline uses Qwen2.5-7B-Instruct in a zero-shot setting with structured prompts, constrained answer types, and schema-based output checking. Invalid or incomplete outputs are handled by a fallback rule rather than being accepted as valid evidence. The evaluation remains limited to 25 pseudo-gold component/version applicability samples from a single firmware image, and should therefore be interpreted as a pilot feasibility study rather than a statistically generalizable benchmark.

The study also does not include a full comparison against existing SBOM/VEX workflows. In particular, direct VEX prompting, evidence-profile-only prompting, rule-based SBOM-CVE matching, and the full checklist-driven method should be compared under the same experimental protocol in future work. The current results are therefore best understood as evidence that the proposed workflow can be executed end-to-end and that its error structure can be inspected, not as proof of deployment-ready automation.

5.5 Deployment Assumptions and Human-in-the-loop Use

The intended users of the pipeline are OT security analysts, vendor PSIRT teams, and supply-chain auditors who need to triage SBOM-derived vulnerability findings. The firmware image, generated SBOM, advisory/CTI sources, operational context, and LLM-generated outputs are treated as separate evidence sources with different trust assumptions. The pipeline does not treat firmware-derived SBOMs as fully trusted ground truth; instead, SBOM evidence is a derived and potentially fallible input whose uncertainty should be propagated to the final interpretation.

Given the current label accuracy, the proposed method should not be deployed as a fully automated VEX decision engine. It is intended to support human analysts by producing evidence-bound candidate states and by making missing evidence explicit. In OT environments, a false not_affected decision can delay mitigation, leave exposed assets unaddressed, and increase operational or safety risk. Therefore, insufficient evidence should conservatively lead to under_investigation rather than a definitive not_affected state.

5.6 LLM Hallucination and Evidence Controls

To reduce hallucination risk, each checklist answer must be grounded in an explicit field of the evidence profile. If the model introduces unsupported evidence, cannot bind an answer to the provided evidence, or produces an inconsistent label-state mapping, the corresponding condition is treated as not_established and should be reviewed by a human analyst. This design turns uncertainty into an explicit output rather than allowing unsupported evidence to drive an affected or not_affected decision.

5.7 Limitations and Future Work

Several limitations remain. First, the current evaluation uses only one firmware image and 25 pseudo-gold component/version applicability samples. Future work should extend the validation to at least three firmware images across different vendors, architectures, and device families. Second, the current study does not test SBOM extraction errors such as BusyBox mis-detection, statically linked OpenSSL omission, missing static libraries, version ambiguity, or custom fork aliasing. These cases are important robustness targets because incorrect SBOM evidence can directly affect component grounding and not_affected decisions.

Third, stronger operational VEX ground truth is required. The current pseudo-gold labels evaluate component presence and affected version range, while runtime usage, reachability, and target-specific advisory evidence remain mostly unresolved. Future work should incorporate richer runtime and network evidence, stronger VEX ground truth sources, and ablation studies that compare direct prompting, evidence-profile-only prompting, and full checklist-conditioned reasoning.

6 Conclusion

This study presented a firmware-SBOM-based approach for checklist-driven, evidence-bound VEX inference in ICS/OT environments. From the perspective of smart manufacturing and industrial digital transformation, the method connects firmware-derived SBOM visibility with contextual VEX interpretation for connected OT assets that may participate in production, remote maintenance, physical-security monitoring, and supply-chain operations. The proposed method combined pseudo-gold firmware-CVE pair construction, ICS-contextual checklist generation, and checklist-based VEX justification extraction.

The experiment showed that the full pipeline could be executed end-to-end on 25 pseudo-gold component/version applicability samples from a single firmware image. Under the measured Qwen2.5-7B-Instruct zero-shot baseline, the method achieved 56.0% label accuracy, 72.0% status accuracy, and 50.5% macro-F1. These results should be interpreted as a pilot feasibility result rather than a statistically generalizable validation or a deployment-ready automated triage result.

The main limitation observed in the case study was not high-level vulnerability reasoning, but component grounding and evidence binding failure. Most errors occurred when the model incorrectly mapped vulnerable or version_not_affected cases to component_not_present. This indicates that improving evidence representation, component-to-asset alignment, and human review of not_affected-relevant outputs is likely to be more important than simply increasing model size.

Overall, this work provides an initial but practical starting point for checklist-driven VEX inference using firmware-derived SBOMs in ICS/OT settings. The method is intended to support auditable analyst decision-making rather than replace human judgment. Future work should expand the dataset across multiple firmware families, incorporate stronger operational VEX ground truth, evaluate SBOM extraction errors, and compare direct prompting, evidence-profile-only prompting, and full checklist-driven reasoning under the same protocol.

Acknowledgment

This work was partly supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government(MSIT) (RS-2025-25394739, Development of Security Enhancement Technology for Industrial Control Systems Based on S/HBOM Supply Chain Protection, 50%) and supported by the Nuclear Safety Research Program through the Korea Foundation Of Nuclear Safety(KoFONS) using the financial resource granted by the Nuclear Safety and Security Commission(NSSC) of the Republic of Korea. (RS-2026-25521970, 50%)

References

[1] NTIA. (2021). The Minimum Elements for a Software Bill of Materials (SBOM).

[2] Bergqvist, Andreas, and Karl-Philip Ble Cato. "Exploring the extension of Software Bill of Materials Usage." LU-CS-EX (2025).
[3] Booth, Harold, Doug Rike, and Gregory A. Witte. "The national vulnerability database (nvd): Overview." (2013).
[4] Cheshkov, Anton, Pavel Zadorozhny, and Rodion Levichev. "Evaluation of chatgpt model for vulnerability detection." arXiv preprint arXiv:2304.07232 (2023).
[5] Gao, Z., Wang, H., Zhou, Y., Zhu, W., & Zhang, C. (2023). How Far Have We Gone in Vulnerability Detection Using Large Language Models (VulBench).
[6] Khare, Avishree, et al. "Understanding the effectiveness of large language models in detecting security vulnerabilities." 2025 IEEE Conference on Software Testing, Verification and Validation (ICST). IEEE, 2025.
[7] Zhang, Chenyuan, et al. "Prompt-enhanced software vulnerability detection using chatgpt." Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings. 2024.
[8] Shestov, Aleksei, et al. "Finetuning large language models for vulnerability detection." IEEE Access (2025).
[9] Yang, Aidan ZH, et al. "Security vulnerability detection with multitask self-instructed fine-tuning of large language models." arXiv preprint arXiv:2406.05892 (2024).
[10] Yao, Shunyu, et al. "React: Synergizing reasoning and acting in language models." The eleventh international conference on learning representations. 2022.
[11] Zemicheal, Tadesse, et al. "LLM agents for vulnerability identification and verification of CVEs." CAMLIS. 2024.
[12] Bi, Tingting, et al. "On the way to sboms: Investigating design issues and solutions in practice." ACM Transactions on Software Engineering and Methodology 33.6 (2024): 1-25.
[13] Stalnaker, Trevor, et al. "Boms away! inside the minds of stakeholders: A comprehensive study of bills of materials for software systems." Proceedings of the 46th IEEE/ACM International Conference on Software Engineering. 2024.
[14] Benedetti, Giacomo, et al. "The impact of SBOM generators on vulnerability assessment in python: A comparison and a novel approach." International Conference on Applied Cryptography and Network Security. Cham: Springer Nature Switzerland, 2025.
[15] Zhou, Li, Marc Dacier, and Charalambos Konstantinou. "A Reality Check on SBOM-based Vulnerability Management: An Empirical Study and A Path Forward." arXiv preprint arXiv:2511.20313 (2025).
[16] Fucci, Davide, et al. "Augmenting software bills of materials with software vulnerability description: A preliminary study on github." Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering. 2025.
[17] Churakova, Yekatierina, Mathias Ekstedt, and Larissa Schmid. "Vexed by VEX tools: Consistency evaluation of container vulnerability scanners." arXiv preprint arXiv:2503.14388 (2025).
[18] Patel, Harshvardhan, Alexander Snit, and Michalis Polychronakis. "Supply Chain Reaction: Enhancing the Precision of Vulnerability Triage using Code Reachability Information." 2025 IEEE Annual Computer Security Applications Conference (ACSAC). IEEE, 2025.
[19] Wen, He. "Vulnerability Assessment of Industrial Control System with an Improved CVSS." arXiv preprint arXiv:2306.08631 (2023).
[20] Tuptuk, Nilufer, and Stephen Hailes. "Identifying vulnerabilities of industrial control systems using evolutionary multiobjective optimisation." Computers & Security 137 (2024): 103593.
[21] Barrère, Martín, et al. "Measuring cyber-physical security in industrial control systems via minimum-effort attack strategies." Journal of information security and applications 52 (2020): 102471.

[22] Ji, Xudong, et al. "A three-stage dynamic assessment framework for industrial control system security based on a method of W-HMM." Sensors 22.7 (2022): 2593.
[23] Makrakis, Georgios Michail, et al. "Vulnerabilities and attacks against industrial control systems and critical infrastructures." arXiv preprint arXiv:2109.03945 (2021).
[24] Hussain, Md Asdaque, Khouloud Samrouth, and Nader Bakir. "A survey on malware attacks in industrial air-gap systems: Md. A. Hussain et al." International Journal of Information Security 24.3 (2025): 146.